

Experiment No.	3
Experiment Title	Regression Analysis using Linear Regression and Regularized Regression Models
GitHub Repository	https://github.com/HarshitaaB-2606
Student Name	<i>Harshitaa B</i>
Register Number	<i>3122247001021</i>
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	<i>16th August 2026</i>

ICS1512 – Machine Learning Algorithms Laboratory

Experiment 3: Regression Analysis

1 Objective

The objectives of this experiment are:

1. To implement Linear Regression for predicting a continuous target variable.
2. To implement Ridge, Lasso, and Elastic Net Regression using regularization techniques.
3. To preprocess a dataset containing numerical and categorical attributes by handling missing values, encoding categorical variables, and scaling numerical variables.
4. To use five-fold cross-validation for model evaluation and Grid Search for hyperparameter selection.
5. To compare Linear Regression, Ridge Regression, Lasso Regression, and Elastic Net Regression using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 score.
6. To analyze the effect of regularization on model complexity, generalization, and regression coefficients.
7. To study the feature-sparsity effect of Lasso Regression and compare the coefficients produced by the different models.
8. To analyze possible overfitting and underfitting through training and test errors and to discuss the bias-variance trade-off.
9. To identify the best-performing regression model for predicting the sanctioned loan amount.

2 Problem Statement

The objective of this experiment is to predict the sanctioned loan amount using applicant-related information. The dataset contains a mixture of numerical and categorical attributes describing the applicant and loan application.

The input consists of applicant and loan-related features such as applicant income, co-applicant income, loan term, credit history, gender, marital status, number of dependents, education, self-employment status, and property area. The output is the continuous variable representing the sanctioned loan amount.

Four regression approaches are evaluated:

1. Linear Regression
2. Ridge Regression
3. Lasso Regression
4. Elastic Net Regression

The provided dataset already contains separate training and testing files. Therefore, the supplied training set is used for model fitting, five-fold cross-validation, and hyperparameter tuning, while the supplied testing set is reserved for final evaluation.

The main objective is not only to obtain a low prediction error, but also to determine whether regularization improves generalization compared with ordinary Linear Regression.

3 Dataset Description

The experiment uses separate training and testing datasets. The target variable is the sanctioned loan amount.

Dataset Property	Description
Dataset Name	Loan-sanction regression dataset (training and testing CSV files)
Dataset Source	https://www.kaggle.com/datasets/altruistdelhite04/loan-prediction-problem-dataset train.csv and test.csv files.
Training Samples	592
Testing Samples	362
Target Variable	Loan amount sanctioned
Numerical Features	ApplicantIncome, CoapplicantIncome, Loan_Amount_Term, Credit_History
Categorical Features	Loan_ID, Gender, Married, Dependents, Education, Self_Employed, Property_Area
Number of Original Features	11

Transformed Features	611 after preprocessing and one-hot encoding
Missing Values	Missing values were handled using imputation within the preprocessing pipeline.
Train-Test Split	A predefined train/test split was supplied as two separate CSV files; no additional random split was performed.
Target Type	Continuous numerical variable

3.1 Feature Description

Feature	Type	Description
ApplicantIncome	Numerical	Income of the primary applicant.
CoapplicantIncome	Numerical	Income of the co-applicant.
Loan_Amount_Term	Numerical	Term/duration associated with the loan.
Credit_History	Numerical	Credit-history indicator.
Loan_ID	Categorical	Identifier associated with the loan application.
Gender	Categorical	Gender of the applicant.
Married	Categorical	Marital status of the applicant.
Dependents	Categorical	Number/category of dependents.
Education	Categorical	Education category of the applicant.
Self_Employed	Categorical	Self-employment status.
Property_Area	Categorical	Property-area category.
Loan Amount	Target	Continuous sanctioned loan amount to be predicted.

Note: The experiment results show that the preprocessing stage produced 611 transformed features. This is expected because categorical variables are converted into one-hot encoded representations.

4 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed before model training to understand the structure and distribution of the data.

4.1 Dataset Overview

The training dataset contains 592 observations and the testing dataset contains 362 observations. The data contains both numerical and categorical variables. Since the target is continuous, this is a regression problem rather than a classification problem.

The numerical features identified during EDA were:

- ApplicantIncome
- CoapplicantIncome
- Loan_Amount_Term
- Credit_History

The categorical features identified were:

- Loan_ID
- Gender
- Married
- Dependents
- Education
- Self_Employed
- Property_Area

4.2 Target Variable Distribution

A histogram was used to inspect the distribution of the sanctioned loan amount.

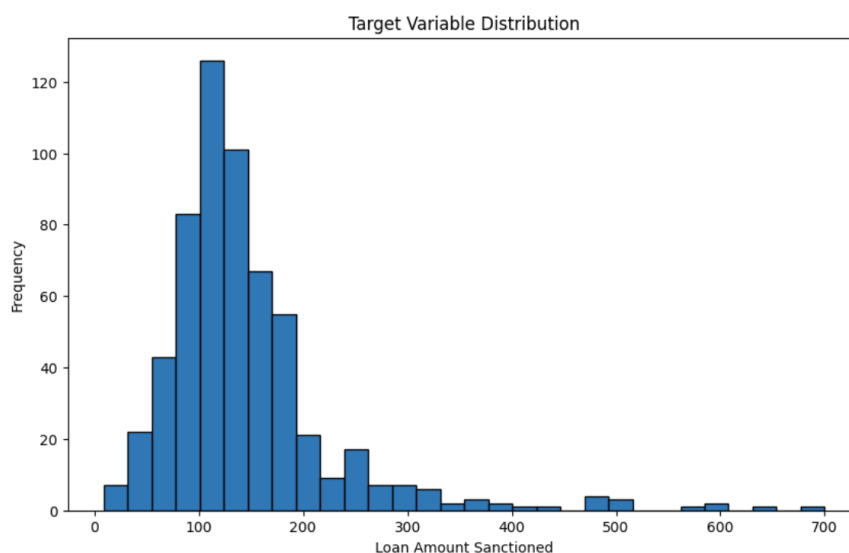


Figure 1: Distribution of the target variable (sanctioned loan amount).

Inference: The target distribution helps identify the spread and concentration of sanctioned loan amounts. A histogram is useful for identifying whether the target is approximately symmetric or whether it contains skewness or extreme observations. The

distribution also provides an initial understanding of the difficulty of the regression task. The model evaluation results indicate that the target contains variation that is only partially explained by the available predictors.

4.3 Numerical Feature Distributions

Histograms were generated for the numerical features.

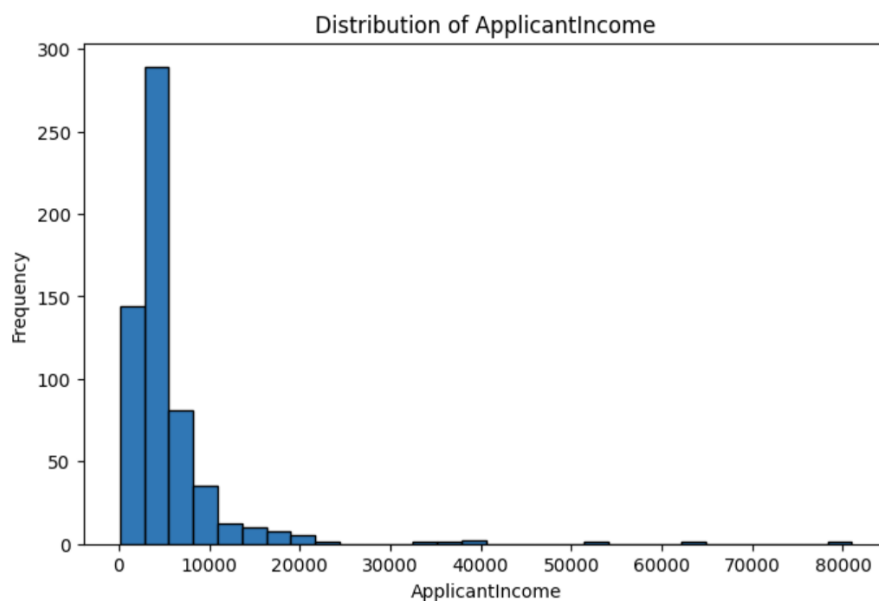


Figure 2: Distributions of numerical input features.

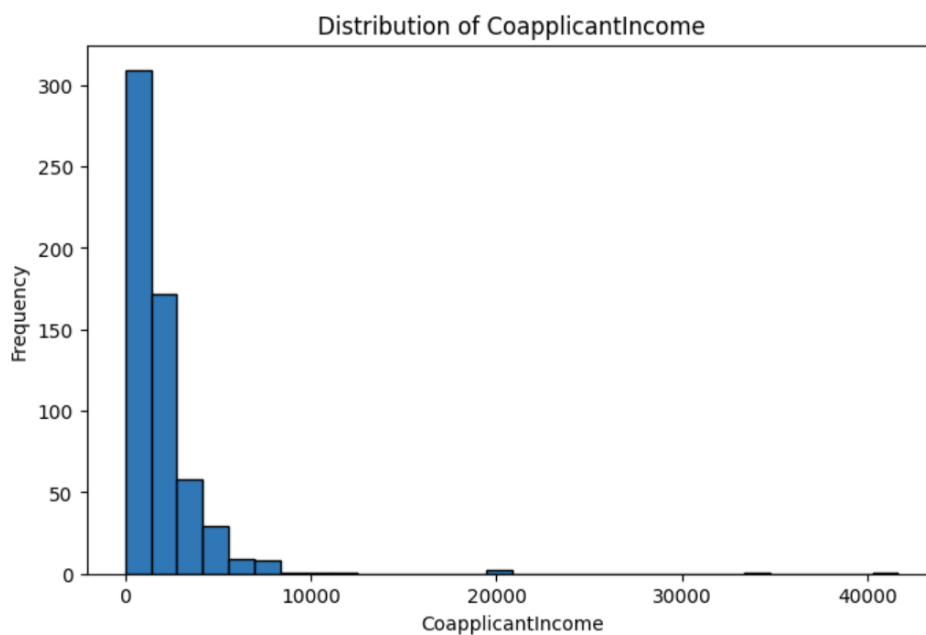


Figure 3: Distributions of numerical input features.

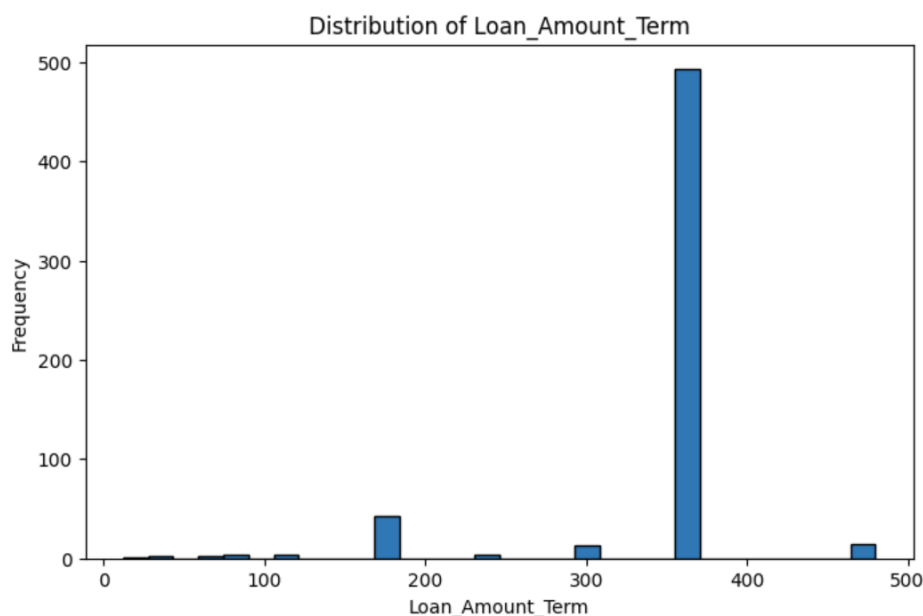


Figure 4: Distributions of numerical input features.

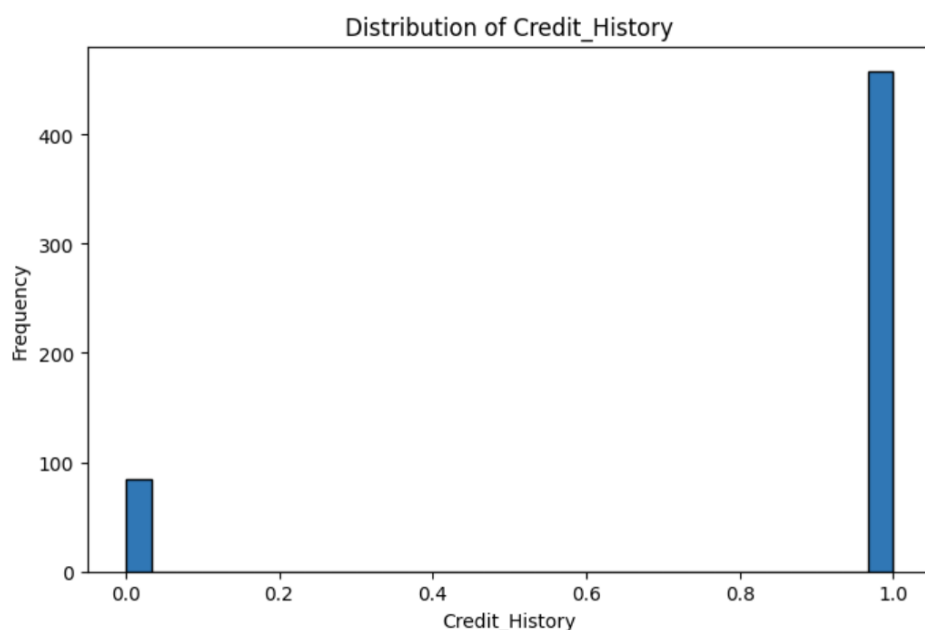


Figure 5: Distributions of numerical input features.

Inference: The numerical-feature distributions show the range and frequency of values for the input variables. Such plots are useful for detecting skewed variables and unusually concentrated observations. These observations motivate the use of median imputation and standardization in the preprocessing pipeline. Standardization is particularly important for regularized regression because the penalty should not be dominated by variables with larger numerical scales.

4.4 Feature versus Target Analysis

Scatter plots were generated between selected numerical features and the target variable.

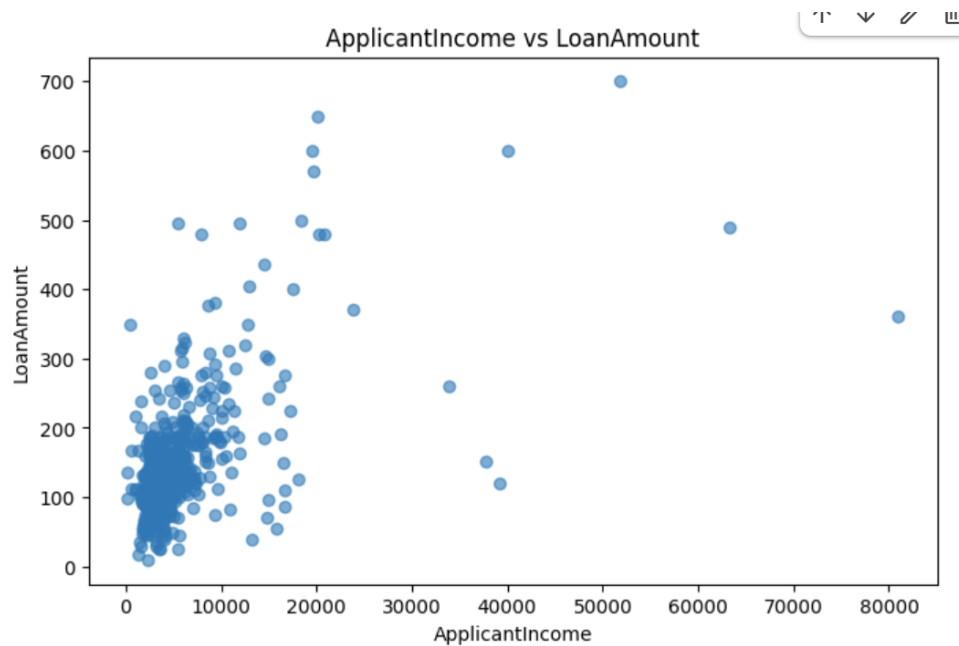


Figure 6: Relationships between selected numerical features and the sanctioned loan amount.

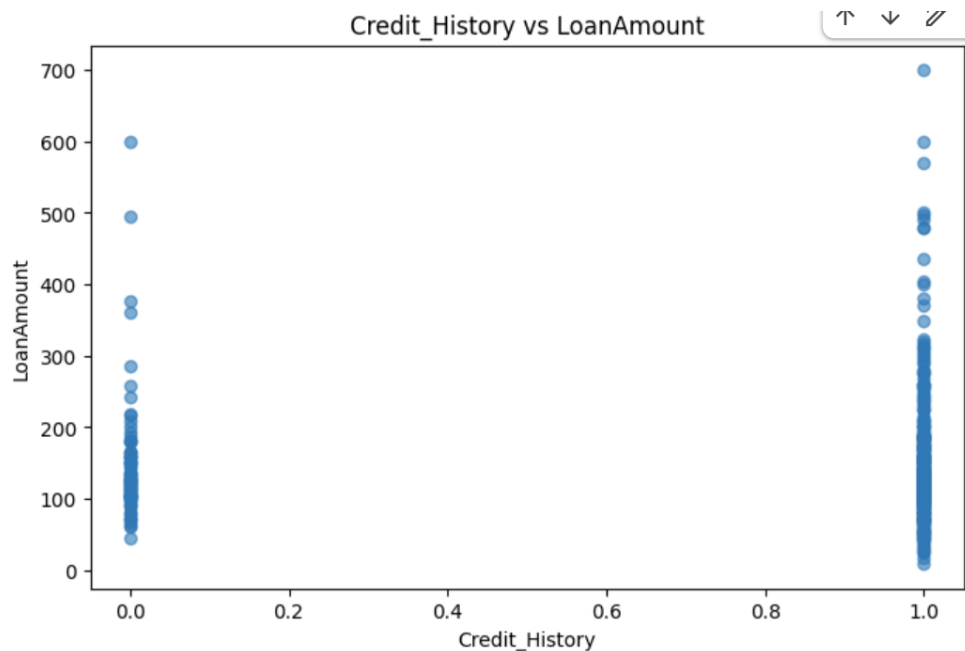


Figure 7: Relationships between selected numerical features and the sanctioned loan amount.

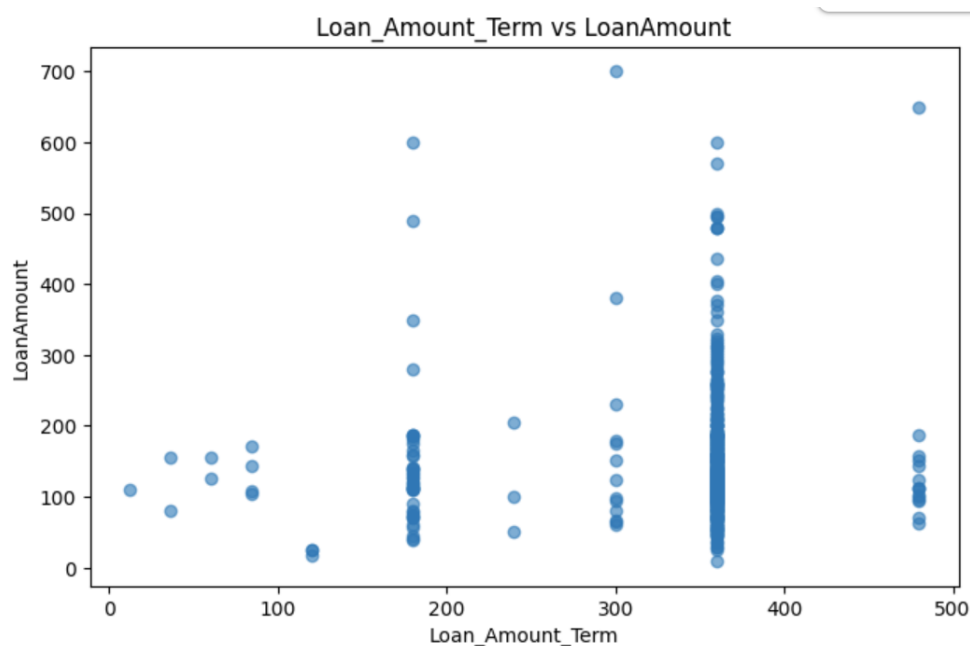


Figure 8: Relationships between selected numerical features and the sanctioned loan amount.

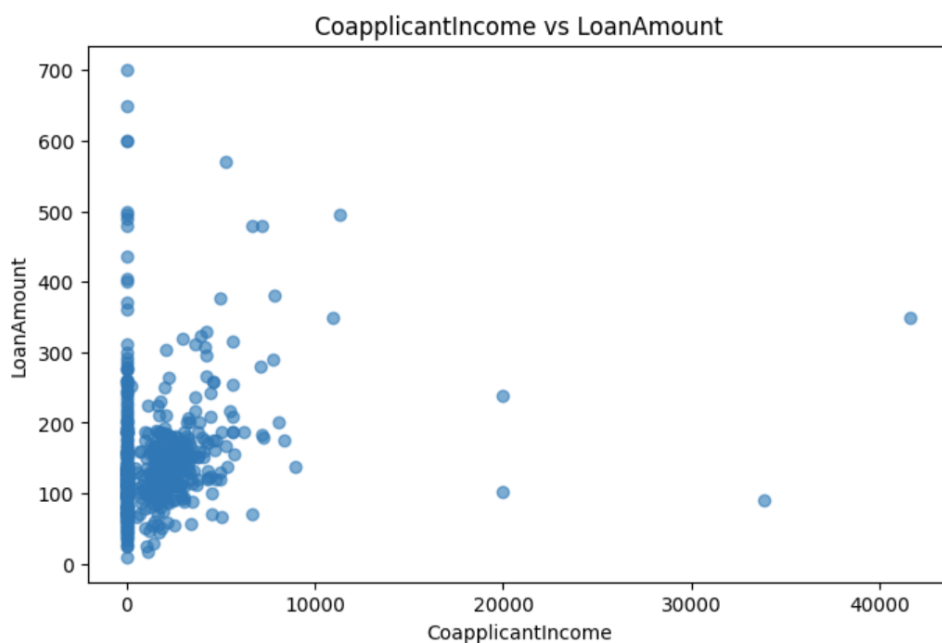


Figure 9: Relationships between selected numerical features and the sanctioned loan amount.

Inference: Feature-versus-target plots provide a visual indication of the relationship between individual predictors and the continuous target. The plots can reveal whether relationships are approximately linear, weak, dispersed, or affected by outliers. Because the regression models use several features simultaneously, a weak individual relationship does not necessarily mean that the feature is useless in the multivariable model.

4.5 Missing Value Analysis

Missing values were inspected in the training data. Instead of removing observations indiscriminately, missing numerical values were replaced using the median and missing categorical values were replaced using the most frequent category. The imputation operations were incorporated into the model pipeline so that preprocessing was learned from the training data.

Inference: Imputation prevents loss of observations and allows the regression algorithms to receive complete numerical matrices. Performing imputation inside the pipeline also prevents information from the test set from being used during model fitting.

5 Data Preprocessing

The preprocessing pipeline was designed separately for numerical and categorical variables.

5.1 Numerical Feature Processing

Numerical features were processed using two steps:

1. Missing values were replaced with the median of the corresponding training feature.
2. Numerical variables were standardized using `StandardScaler`.

Standardization transforms a feature x according to:

$$z = \frac{x - \mu}{\sigma}$$

where μ is the training-set mean and σ is the training-set standard deviation.

Scaling is particularly important for Ridge, Lasso, and Elastic Net because their regularization penalties depend on coefficient magnitude.

5.2 Categorical Feature Processing

Categorical variables were processed using:

1. Most-frequent imputation for missing categorical values.
2. One-hot encoding to convert categorical variables into numerical indicator variables.

One-hot encoding allows regression algorithms to use categorical information without imposing an artificial numerical ordering on the categories.

5.3 Train-Test Handling

The dataset was already divided into separate training and testing CSV files. Therefore, no additional `train_test_split()` operation was used.

The training dataset was used for:

- preprocessing fitting,
- model training,
- five-fold cross-validation,
- hyperparameter tuning.

The testing dataset was kept for final performance evaluation.

This separation is important because the test set should represent unseen observations when reporting the final model performance.

6 Machine Learning Algorithm

6.1 Linear Regression

Linear Regression models the target as a linear combination of the input features:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

The coefficients are estimated by minimizing the residual sum of squares:

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Linear Regression was used as the baseline model.

Advantages:

- Simple and interpretable.
- Computationally efficient.
- Provides a useful baseline for comparing regularized models.

Limitations:

- Can be sensitive to multicollinearity.
- Can have high variance when many correlated features are present.
- Does not explicitly control model complexity.

6.2 Ridge Regression

Ridge Regression adds an L2 penalty to the ordinary least-squares objective:

$$\min_{\beta} \left[\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^p \beta_j^2 \right]$$

The parameter α controls the strength of regularization.

A larger α produces stronger coefficient shrinkage. In this experiment, the best Ridge value was $\alpha = 100$.

Advantages:

- Reduces coefficient magnitude.
- Helps control variance.
- Can improve generalization in the presence of correlated predictors.

Limitation: Ridge normally does not force coefficients exactly to zero, so it does not provide the same direct feature-selection effect as Lasso.

6.3 Lasso Regression

Lasso uses an L1 penalty:

$$\min_{\beta} \left[\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^p |\beta_j| \right]$$

The L1 penalty can force some coefficients exactly to zero.

The best value obtained through Grid Search was:

$$\alpha = 1$$

Lasso produced 602 zero coefficients out of 611 transformed features, corresponding to approximately:

$$\frac{602}{611} \times 100 = 98.53\%$$

zero coefficients.

Advantages:

- Performs regularization.
- Performs implicit feature selection.

- Produces sparse models.

Limitation: Excessive sparsity can remove potentially useful predictors and may increase bias.

6.4 Elastic Net Regression

Elastic Net combines L1 and L2 regularization:

$$\min_{\beta} \left[\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \alpha \left(\rho \sum_{j=1}^p |\beta_j| + (1 - \rho) \sum_{j=1}^p \beta_j^2 \right) \right]$$

where α controls the overall regularization strength and ρ (implemented as `l1_ratio`) controls the mixture of L1 and L2 penalties.

The best configuration was:

$$\alpha = 1, \quad l1_ratio = 0.8$$

Elastic Net therefore provided a combination of coefficient shrinkage and sparsity.

7 Experimental Setup

Python Version	Google Colab Python environment
Scikit-Learn	Scikit-learn library
IDE	Google Colab
Operating System	Cloud-based Linux environment
Hardware	Google Colab runtime
Cross-validation	5-fold K-Fold CV
Random State	42

The exact Python and Scikit-learn version numbers were not recorded in the supplied experiment output. They should be inserted from the Colab environment if required.

8 Hyperparameter Selection

Five-fold cross-validation and Grid Search were used for selecting regularization parameters.

Table 3: Hyperparameter search spaces

Model	Parameter	Search Values
Ridge	α	0.01, 0.1, 1, 10, 100
Lasso	α	0.001, 0.01, 0.1, 1, 10
Elastic Net	α	0.01, 0.1, 1, 10
Elastic Net	<code>l1_ratio</code>	0.2, 0.5, 0.8

Table 4: Best hyperparameters obtained from Grid Search

Model	Best Parameters	Best CV R^2
Ridge	$\alpha = 100$	0.296446
Lasso	$\alpha = 1$	0.269309
Elastic Net	$\alpha = 1$, <code>l1_ratio</code> = 0.8	0.291913

The best cross-validation score among the tuned models was obtained by Ridge Regression with $R^2 = 0.296446$. However, the final model selection was based on the performance on the held-out test set, where Elastic Net achieved the highest R^2 .

9 Results

9.1 Five-Fold Cross-Validation Results

Table 5: Five-fold cross-validation performance

Model	MAE	MSE	RMSE	R^2
Linear Regression	42.7399	5362.2118	73.2271	0.2629
Ridge Regression	43.3919	5026.3732	70.8969	0.2964
Lasso Regression	42.5940	5323.2378	72.9605	0.2693
Elastic Net	43.5942	5061.0426	71.1410	0.2919

9.2 Final Test-Set Results

The final evaluation was performed using the supplied testing dataset containing 362 observations.

Table 6: Final test-set performance

Model	MAE	MSE	RMSE	R^2	Time (s)
Linear Regression	36.0467	2728.8873	52.2388	0.2734	0.2250
Ridge Regression	35.7577	2606.4552	51.0535	0.3060	5.4762
Lasso Regression	35.4400	2660.1341	51.5765	0.2917	8.6063
Elastic Net	35.7653	2587.4734	50.8672	0.3110	2.5264

9.3 Interpretation of Results

Elastic Net Regression achieved the highest final test-set R^2 score of 0.3110. It also achieved the lowest MSE of 2587.4734 and the lowest RMSE of 50.8672. Therefore, Elastic Net was selected as the best overall model.

Lasso Regression achieved the lowest MAE of 35.4400. This means that Lasso produced the smallest average absolute prediction error. However, its R^2 score was 0.2917, lower than the 0.3110 obtained by Elastic Net.

Ridge Regression achieved the second-highest test R^2 score of 0.3060. It also performed better than Linear Regression on MSE, RMSE, and R^2 , demonstrating the benefit of regularization.

The test-set results show that regularization improved generalization compared with the baseline Linear Regression model.

10 Result Visualization

10.1 Predicted versus Actual Values

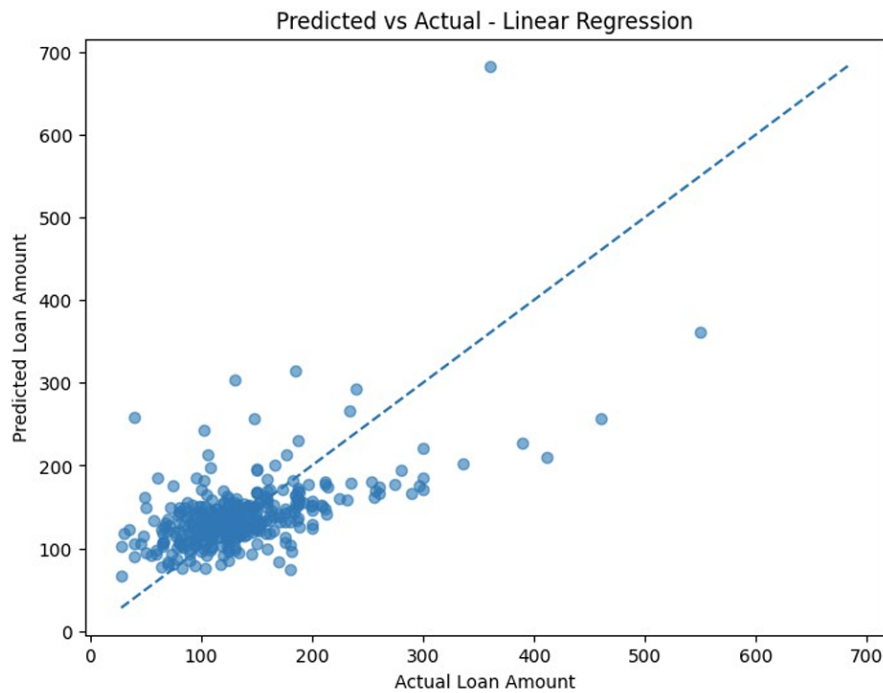


Figure 10: Predicted versus actual loan amounts for the regression models.

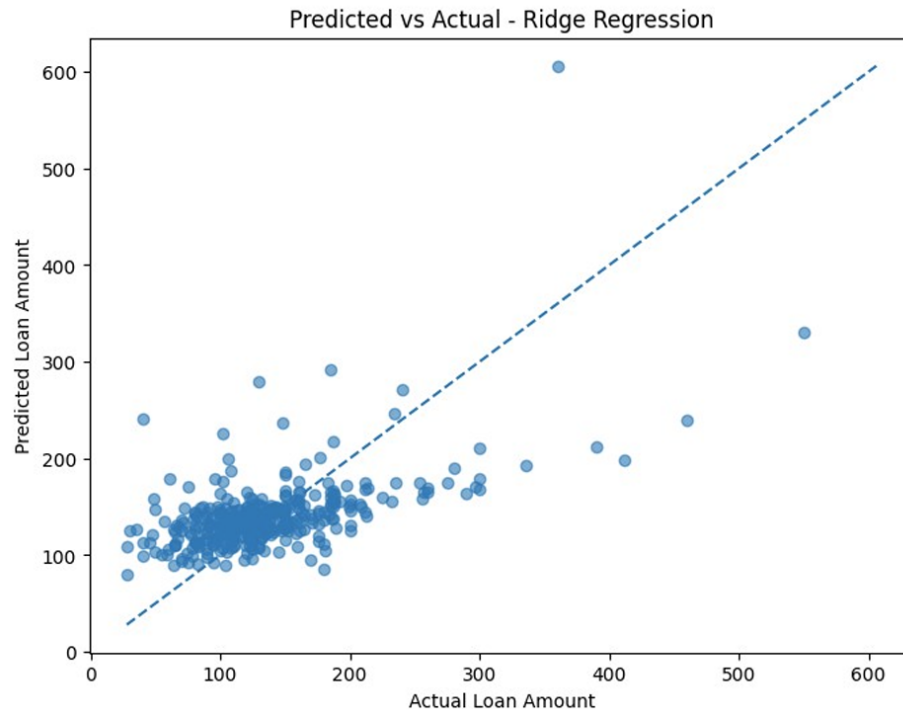


Figure 11: Predicted versus actual loan amounts for the regression models.

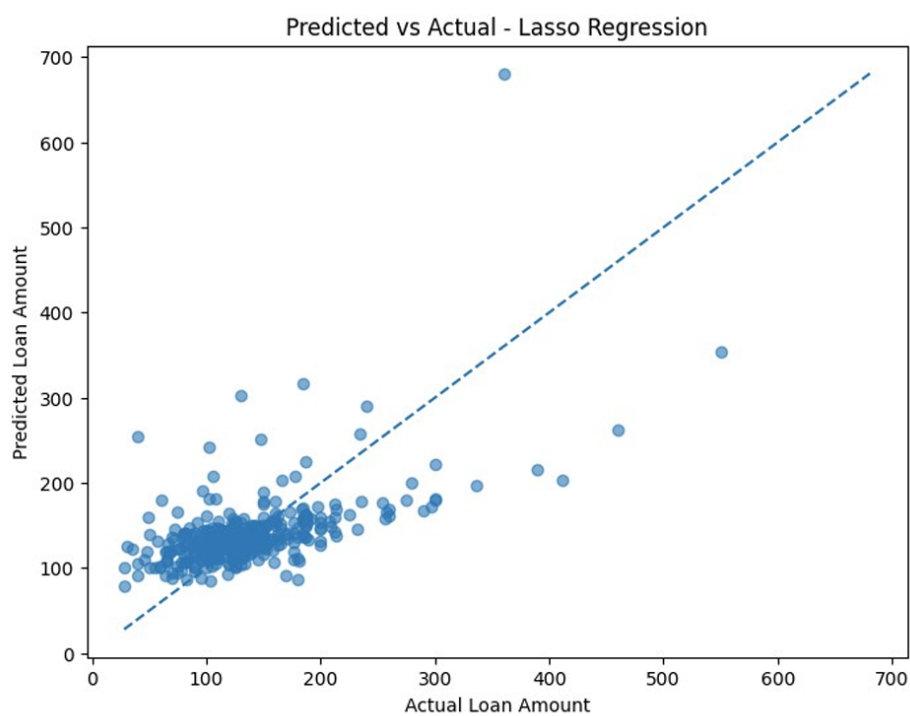


Figure 12: Predicted versus actual loan amounts for the regression models.

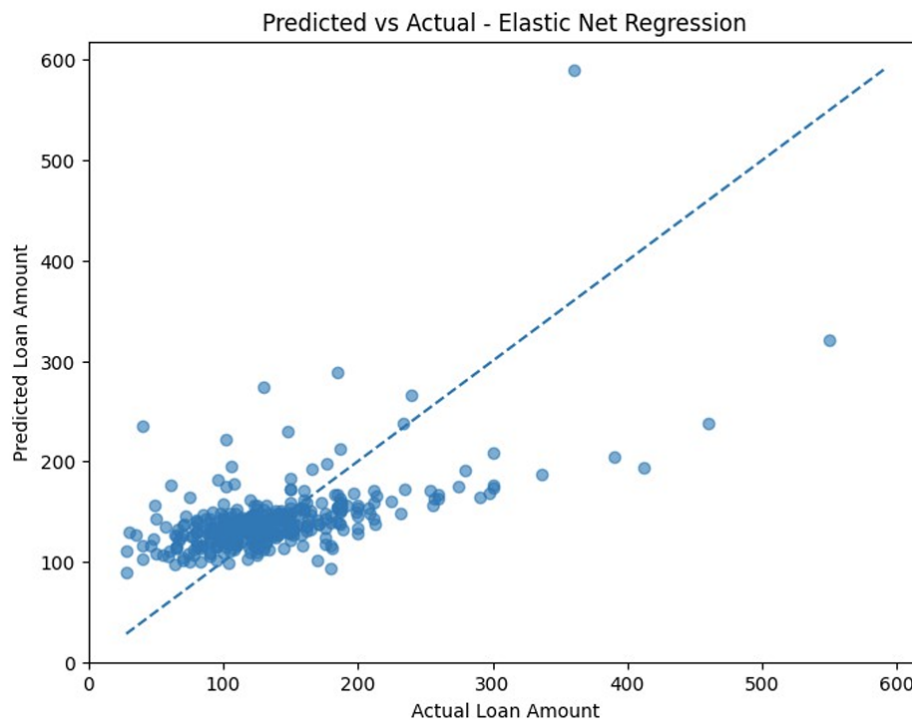


Figure 13: Predicted versus actual loan amounts for the regression models.

Inference: The predicted-versus-actual plot is used to evaluate how closely model predictions follow the actual target values. Points close to the diagonal reference line indicate more accurate predictions. The dispersion of points around the line indicates prediction error. Elastic Net produced the strongest overall test performance according to R^2 , MSE, and RMSE.

10.2 Residual Analysis

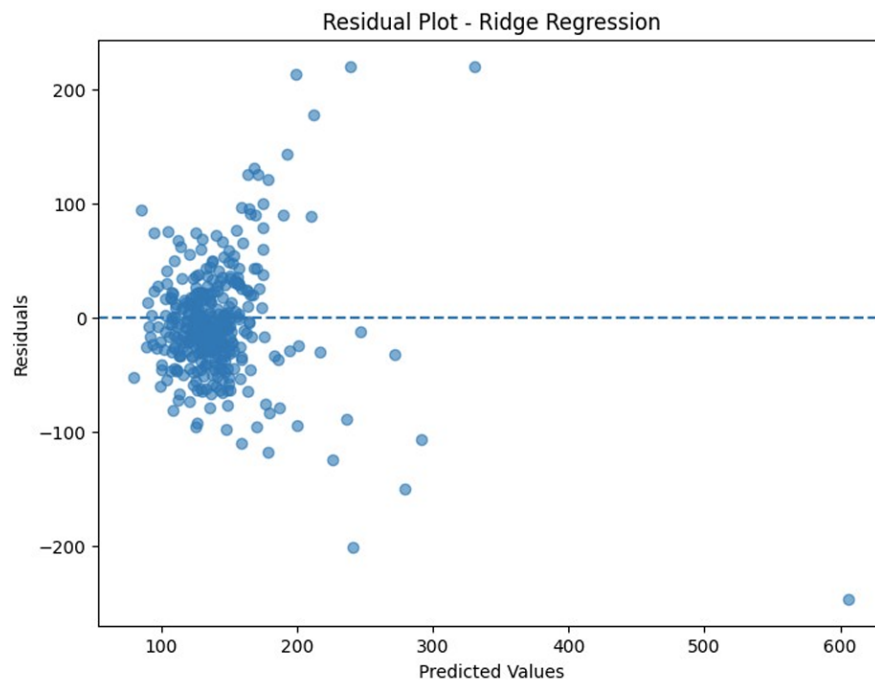


Figure 14: Residual plots for the regression models.

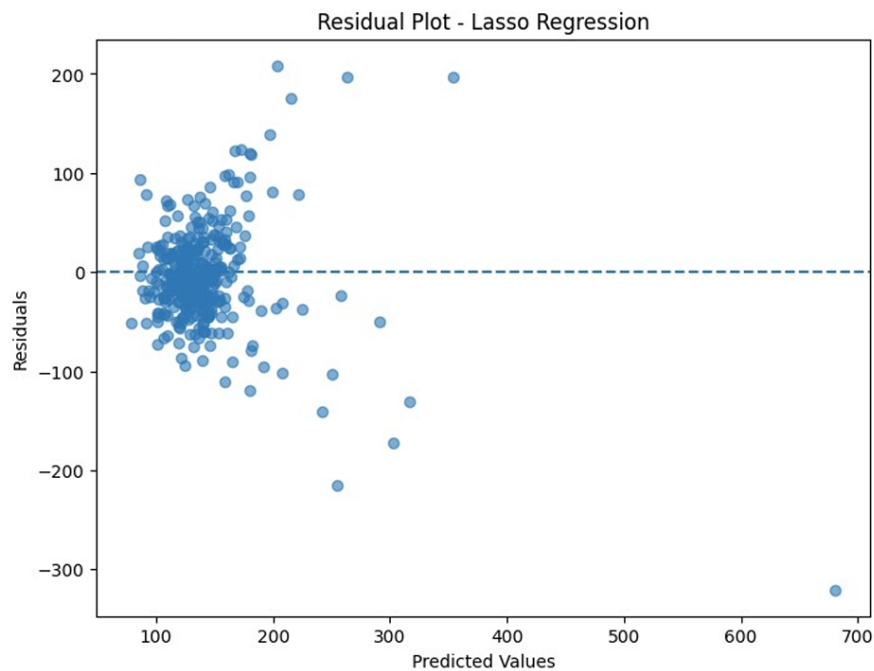


Figure 15: Residual plots for the regression models.

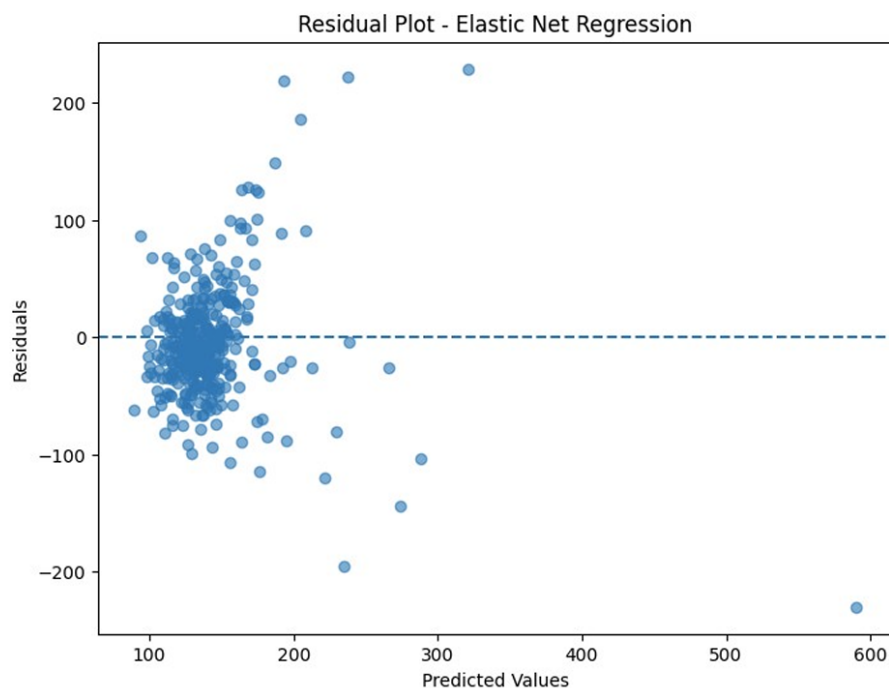


Figure 16: Residual plots for the regression models.

Inference: Residual plots show the difference between observed and predicted values. A desirable residual pattern is approximately centered around zero without a strong systematic structure. Systematic patterns can indicate model inadequacy or non-linearity. Residual analysis therefore provides a complementary assessment beyond numerical metrics.

10.3 Training Error versus Test Error

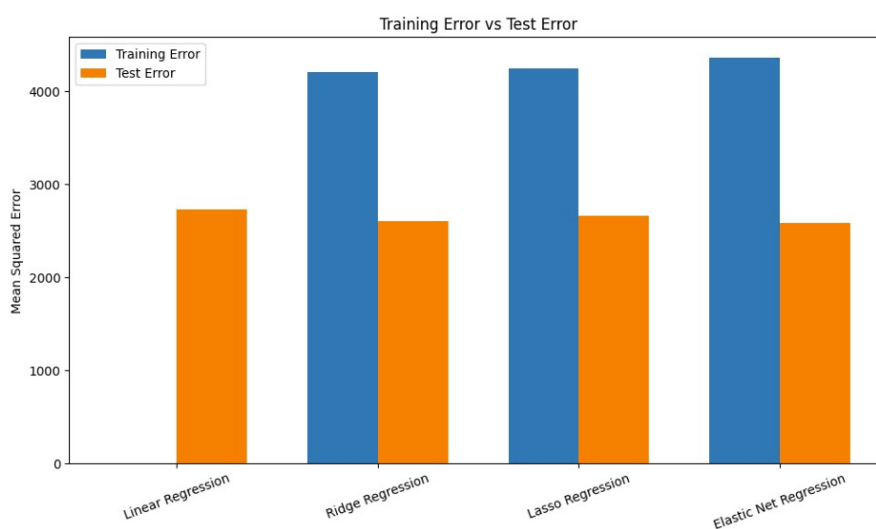


Figure 17: Comparison of training and test mean squared error.

Inference: Linear Regression obtained an extremely small training MSE but a larger test MSE, showing that it fits the training observations very closely. The regularized models have larger training errors but lower test errors. This pattern is consistent with regularization reducing variance and improving generalization.

10.4 R^2 Score Comparison

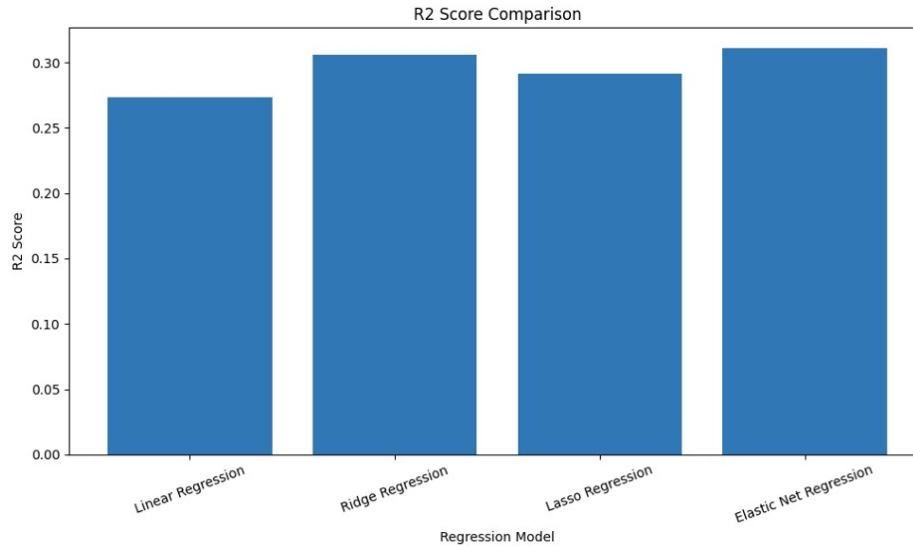


Figure 18: R^2 score comparison of the regression models.

Inference: Elastic Net achieved the highest test R^2 score of 0.3110, followed by Ridge with 0.3060. Lasso achieved 0.2917, while Linear Regression achieved 0.2734. The results indicate that regularization improves the proportion of target variation explained by the model on unseen data.

10.5 Coefficient Comparison

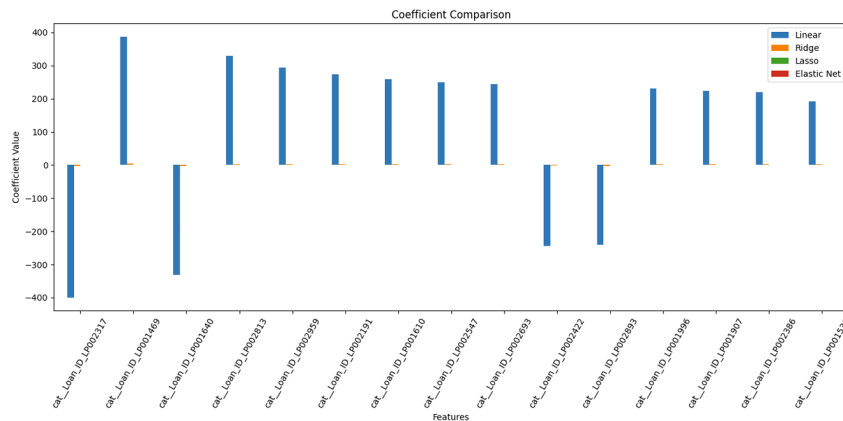


Figure 19: Comparison of regression coefficients across models.

Inference: The coefficient comparison demonstrates the effect of regularization on model parameters. Ridge shrinks coefficients toward zero, while Lasso can set coefficients exactly to zero. Elastic Net combines both effects. This analysis helps explain how regularization controls model complexity.

11 Discussion

11.1 Model Comparison

The four models provide different approaches to controlling model complexity.

Linear Regression serves as the baseline. It obtained a test R^2 of 0.2734. Its very low training MSE indicates that it can fit the training data extremely closely, but its test performance is weaker than the regularized models.

Ridge Regression obtained a test R^2 of 0.3060. The L2 penalty reduced coefficient magnitude and improved test generalization. Its best regularization strength was $\alpha = 100$.

Lasso Regression obtained a test R^2 of 0.2917 and the lowest MAE of 35.4400. It also produced a highly sparse model, with 602 of 611 transformed coefficients equal to zero.

Elastic Net Regression achieved the best overall test performance. Its test R^2 was 0.3110, while its MSE and RMSE were also the lowest among the four models. Its optimal parameters were $\alpha = 1$ and $l1_ratio = 0.8$.

11.2 Overfitting and Underfitting

The training and test errors demonstrate why regularization is useful. Linear Regression has a training MSE close to zero, whereas its test MSE is 2728.8873. This large difference indicates that the unregularized model fits the training data very closely and does not generalize as well as the regularized alternatives.

The regularized models accept a higher training error in exchange for better test performance. Ridge, Lasso, and Elastic Net all have lower test MSE than Linear Regression. This indicates that the regularization penalties help reduce variance and improve generalization.

The results do not provide evidence of severe underfitting in the sense of identical poor training and test performance. Instead, the primary observation is that the unregularized model benefits from regularization when evaluated on unseen data.

11.3 Bias–Variance Trade-off

Regularization introduces a controlled amount of bias in exchange for a reduction in variance.

Linear Regression has less constraint on its coefficients and can therefore achieve extremely low training error. However, this flexibility can lead to higher variance.

Ridge introduces an L2 penalty and reduces coefficient magnitude. This generally reduces variance while retaining all predictors.

Lasso introduces an L1 penalty and performs feature selection. Its strong sparsity result shows that it removed most transformed coefficients.

Elastic Net combines L1 and L2 penalties. In this experiment, this combination produced the best final test-set performance, suggesting that it provided a favorable balance between bias and variance for this dataset.

11.4 Feature Sparsity

Lasso produced 602 zero coefficients out of 611 transformed features. The resulting sparsity was:

$$98.53\%$$

This demonstrates that L1 regularization can substantially simplify a model by removing many transformed predictors. However, such high sparsity should be interpreted carefully because removing too many coefficients can also discard useful information.

11.5 Strengths

The experiment has several strengths:

- Four different regression strategies were compared.
- A predefined test set was kept separate from model training.
- Five-fold cross-validation was used for model selection.
- Grid Search was used for systematic hyperparameter tuning.
- Both numerical and categorical features were handled through a unified preprocessing pipeline.
- Multiple evaluation metrics were used rather than relying on a single score.
- Coefficient behavior and Lasso sparsity were analyzed.

11.6 Limitations

The models explain only about 27–31% of the variance in the test target, as indicated by the R^2 values. This suggests that the available features do not fully explain variation in sanctioned loan amounts.

Another limitation is that the dataset contains an identifier-type variable, `Loan_ID`, which was treated as a categorical feature in the supplied implementation. Identifier variables generally have limited predictive meaning and can create a large number of one-hot

encoded columns. The resulting 611 transformed features also contribute to the high dimensionality of the model.

The experiment evaluates linear and regularized linear models. Non-linear relationships may not be captured adequately by these models.

11.7 Possible Improvements

Possible improvements include:

1. Removing identifier variables such as `Loan_ID` if they do not carry meaningful predictive information.
2. Creating domain-based features such as total household income.
3. Testing non-linear regression methods for comparison.
4. Investigating outliers and highly skewed variables.
5. Using additional feature engineering based on the meaning of the loan application variables.
6. Performing broader hyperparameter searches if computational resources permit.

12 Conclusion

This experiment implemented and compared Linear Regression, Ridge Regression, Lasso Regression, and Elastic Net Regression for predicting sanctioned loan amounts.

The preprocessing pipeline handled missing values, encoded categorical variables, and standardized numerical variables. Five-fold cross-validation and Grid Search were used to select regularization parameters for Ridge, Lasso, and Elastic Net.

The final test-set results showed that Elastic Net was the best overall model, achieving an R^2 score of **0.3110**, MSE of **2587.4734**, and RMSE of **50.8672**. Ridge was the second-best model based on R^2 , with a score of 0.3060. Lasso achieved the lowest MAE of 35.4400 and demonstrated strong feature sparsity, with 98.53% of transformed coefficients reduced to zero.

The experiment therefore demonstrates that regularization can improve generalization compared with ordinary Linear Regression. Elastic Net provided the best overall balance of prediction performance and regularization for the supplied dataset and was selected as the final model.

13 References

- [1] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.

- [2] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [3] Scikit-learn Developers, “Linear Models,” Scikit-learn Documentation. [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html
- [4] Scikit-learn Developers, “Preprocessing data,” Scikit-learn Documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/preprocessing.html>
- [5] Scikit-learn Developers, “Model selection and evaluation,” Scikit-learn Documentation. [Online]. Available: https://scikit-learn.org/stable/model_selection.html
- [6] Scikit-learn Developers, “Metrics and scoring: quantifying the quality of predictions,” Scikit-learn Documentation. [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html

A Appendix A: Important Experimental Results

Table 7: Summary of final model selection

Quantity	Value
Best Model	Elastic Net Regression
Best Test R^2	0.3110108
Best Test MSE	2587.4734
Best Test RMSE	50.8672
Best Test MAE	35.7653
Best Elastic Net α	1
Best Elastic Net $l1_ratio$	0.8
Transformed Features	611
Lasso Zero Coefficients	602
Lasso Sparsity	98.53%

B Appendix B: Core Python Implementation

```

1 # Separate features and target
2 X_train = train_df.drop(columns=[target_column])
3 y_train = train_df[target_column]
4
5 X_test = test_df.drop(columns=[target_column])
6 y_test = test_df[target_column]
7
8 # Numerical preprocessing
9 numeric_transformer = Pipeline(

```

```
10     steps=[
11         ("imputer", SimpleImputer(strategy="median")),
12         ("scaler", StandardScaler())
13     ]
14 )
15
16 # Categorical preprocessing
17 categorical_transformer = Pipeline(
18     steps=[
19         ("imputer",
20          SimpleImputer(strategy="most_frequent")),
21         ("encoder",
22          OneHotEncoder(handle_unknown="ignore",
23                        sparse_output=False))
24     ]
25 )
26
27 # Combined preprocessing
28 preprocessor = ColumnTransformer(
29     transformers=[
30         ("num", numeric_transformer, numeric_features),
31         ("cat", categorical_transformer, categorical_features)
32     ]
33 )
34
35 # Elastic Net model
36 elastic_pipeline = Pipeline(
37     steps=[
38         ("preprocessor", preprocessor),
39         ("model", ElasticNet(max_iter=10000))
40     ]
41 )
42
43 # Grid Search
44 elastic_params = {
45     "model__alpha": [0.01, 0.1, 1, 10],
46     "model__l1_ratio": [0.2, 0.5, 0.8]
47 }
48
49 elastic_grid = GridSearchCV(
50     elastic_pipeline,
51     elastic_params,
52     cv=5,
53     scoring="r2",
54     n_jobs=-1
55 )
56
57 elastic_grid.fit(X_train, y_train)
58
59 # Final predictions
60 elastic_test_pred = elastic_grid.predict(X_test)
```